



Tecnológico de Monterrey

Diagrama de Despliegue

Equipo “Financial Sentinel”

Isabella Montiel Reyes - A01278286

Iker Rodríguez - A01712814

Axel Orlando Arenas Vergara - A01278654

Construcción de software y toma de decisiones (Gpo 501)

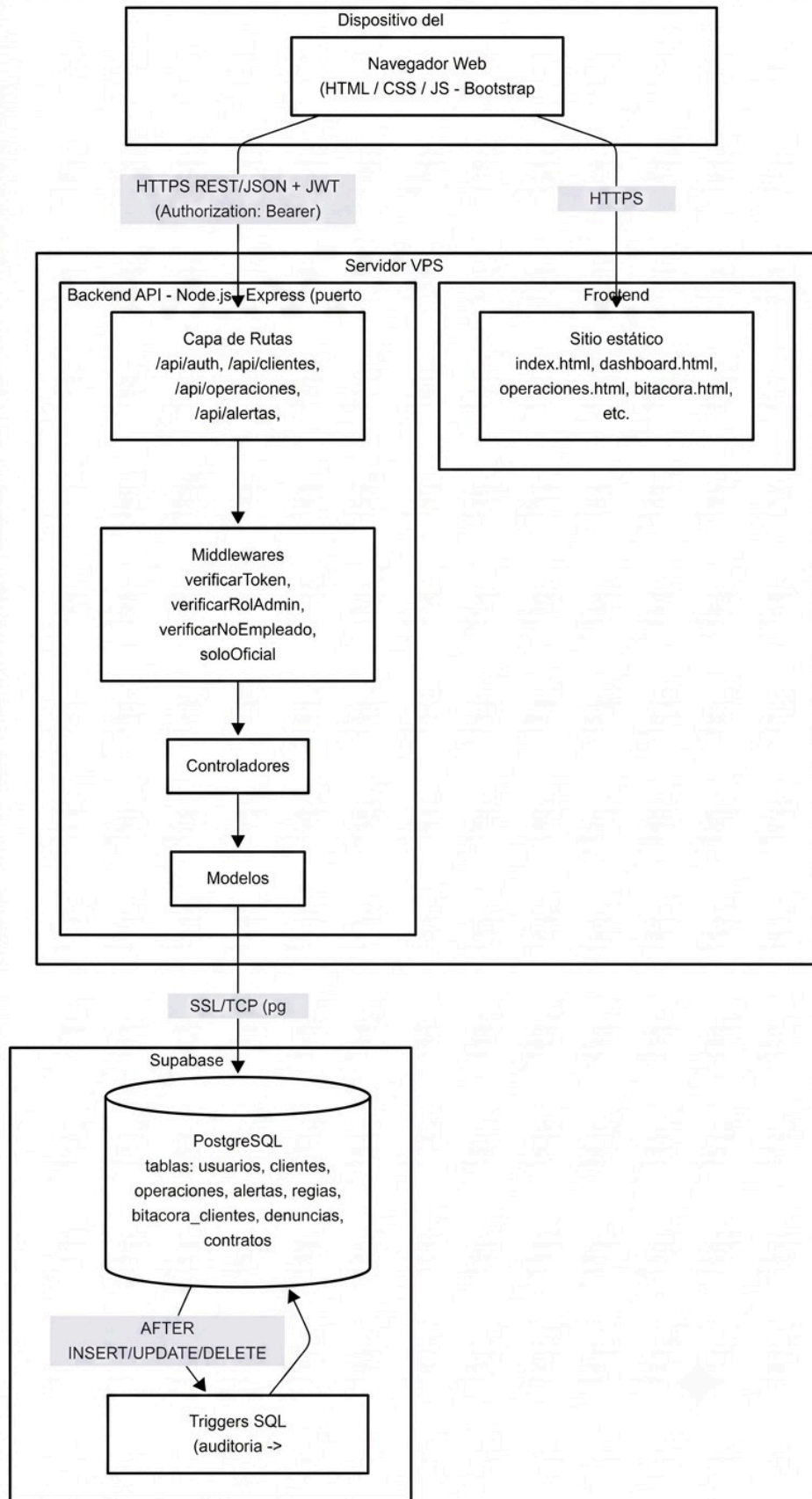
Profesores:

Enrique Alfonso Calderón Balderas

Denisse Lizbeth Maldonado Flores

Alejandro Fernández Vilchis

12/06/2026



El sistema de despliegue de Financial Sentinel sigue una arquitectura cliente-servidor de tres capas, todas alojadas en infraestructura separada pero conectadas vía HTTPS/SSL. A continuación te explico cada componente y cómo interactúan.

Capa cliente (navegador del usuario)

Cualquier usuario (Administrador, Oficial de Cumplimiento, Visor o Empleado) accede al sistema desde su navegador web. Ahí se ejecuta todo el frontend: HTML, CSS y JavaScript vanilla con Bootstrap 5. El navegador hace dos tipos de peticiones distintas: una para cargar los archivos estáticos de la interfaz (las páginas como dashboard.html, operaciones.html, bitacora.html, etc.) y otra para comunicarse con la API mediante peticiones REST en formato JSON.

Capa de servidor (VPS en sslip.io)

Aquí viven dos cosas que conviven en el mismo servidor:

El frontend estático simplemente se sirve como archivos; no tiene lógica de servidor propia, es lo que el navegador descarga y ejecuta localmente.

El backend es una API construida con Node.js y Express, corriendo en el puerto 3000 (definido en server.js). Esta API está organizada en capas: primero las rutas (clientRoutes, authRoutes, bitacoraRoutes, etc.), que reciben la petición y la dirigen al middleware correspondiente. Los middlewares de authMiddleware.js son el punto de control de seguridad: verificarToken valida que la petición tenga un JWT válido en el header Authorization, y luego middlewares como verificarRolAdmin, verificarNoEmpleado o soloOficial verifican que el rol del usuario tenga permiso para esa acción específica (por ejemplo, la bitácora solo la puede ver oficial@fsst.com). Si todo pasa, la petición llega al controlador correspondiente, que contiene la lógica de negocio, y este a su vez llama al modelo, que es quien ejecuta las consultas SQL usando el pool de conexiones de pg.

Un detalle importante de seguridad/configuración: el servidor tiene CORS configurado para aceptar peticiones únicamente desde el dominio del frontend en sslip.io, lo cual amarra ambas piezas (frontend y backend) como un mismo "origen" confiable.

Capa de base de datos (Supabase Cloud)

La base de datos PostgreSQL no vive en el mismo servidor, sino que está administrada por Supabase como un servicio externo. La conexión se hace mediante una cadena DATABASE_URL con SSL habilitado (rejectUnauthorized: false), lo que significa que el backend se conecta de forma cifrada pero sin validar estrictamente el certificado, algo común en conexiones a Supabase desde backends self-hosted.

Dentro de esta base de datos están las tablas principales del sistema (usuarios, clientes, operaciones, alertas, reglas, bitacora_clientes, denuncias, contratos), pero también hay lógica que vive directamente en la base de datos: los triggers SQL. Estos triggers se disparan automáticamente cuando hay un INSERT, UPDATE o DELETE sobre la tabla clientes, y generan de forma autónoma un registro en bitacora_clientes con los datos anteriores y nuevos. Esto es clave porque significa que la auditoría no depende de que el backend "se acuerde" de registrarla en cada controlador; es la base de datos misma la que garantiza la trazabilidad.

Flujo general resumido

1. El navegador carga la interfaz estática desde el VPS.
2. Cada acción del usuario (login, consultar operaciones, dictaminar alertas, etc.) genera una petición HTTPS con JSON hacia la API.
3. La API valida el JWT y el rol, ejecuta la lógica correspondiente y consulta o modifica la base de datos en Supabase vía conexión SSL.
4. Si la operación afecta la tabla clientes, un trigger en Supabase genera automáticamente el registro de bitácora, sin intervención del backend.
5. La respuesta regresa como JSON al frontend, que actualiza la interfaz sin recargar la página.